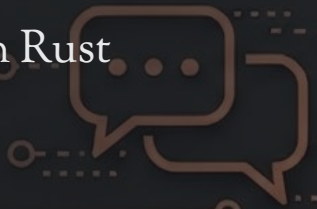
Corso di Programmazione di Sistema 2025/2026

Progetto Ruggine

Sistema di Messaggistica Real-Time in Rust



Politecnico di Torino

Gruppo 50

Alessandro Annecchini · Riccardo Marconi · Francesco Palmisani · Giosuè Pinto

Navigation

Agenda

01 **Introduzione**

Panoramica del progetto e analisi dei requisiti funzionali e non funzionali del sistema.

02 **Architettura**

Struttura a 3 livelli: Frontend, Backend e Database con le tecnologie adottate.

03 **Frontend React**

Componenti, pagine principali e gestione dello stato globale con WebSocket Context.

04 **Backend Rust**

Pattern Controller-Service-Repository, autenticazione JWT e gestione messaggi real-time.

05 **Database**

Schema relazionale, tabelle principali e meccanismi di sicurezza e integrità.

06 **Esperienza Utente**

Funzionalità principali, interfaccia intuitiva e sistema di notifiche in tempo reale.

07 **Sviluppi Futuri**

Microservizi, containerizzazione, notifiche push e miglioramenti alla sicurezza.

Politecnico di Torino

01

Introduzione

Panoramica del progetto e requisiti



Project Overview

Il Progetto Ruggine

Obiettivo del Sistema

Realizzare un'applicazione **client/server** per la gestione di una chat testuale che consenta la comunicazione tra utenti e la creazione di gruppi per la condivisione di messaggi in tempo reale.

Il sistema è progettato per garantire prestazioni elevate , sicurezza robusta e scalabilità , sfruttando le caratteristiche uniche del linguaggio Rust.

Team di Sviluppo - Gruppo 50

AA — Alessandro Annecchini — s343760

RM — Riccardo Marconi — s342227

FP — Francesco Palmisani — s343429

GP — Giosuè Pinto — s342711

Metriche Progetto

Dimensione Windows **6.73 MB**

Dimensione macOS **7.9 MB**

Compatibilità **Multipiattaforma**

Tecnologie Core

Rust · React · WebSocket · SQLite · Axum · Tokio

Requirements Analysis

Requisiti Funzionali e Non Funzionali

Requisiti Funzionali

Iscrizione Utente
Registrazione con username e password

Login Sicuro
Autenticazione con JWT token

Scambio Messaggi
Comunicazione real-time via WebSocket

Creazione Gruppi
Chat di gruppo con inviti

Gestione Inviti
Sistema inviti con accettazione/rifiuto

Logging CPU
Monitoraggio ogni 2 minuti

Requisiti Non Funzionali

Prestazioni
Basso consumo CPU, eseguibile compatto

Sicurezza
Argon2 hashing, JWT token, SQL injection protection

Scalabilità
Architettura per gestire crescita utenti

Usabilità
Interfaccia semplice e intuitiva

Portabilità
Windows, Linux, macOS Android, iOS

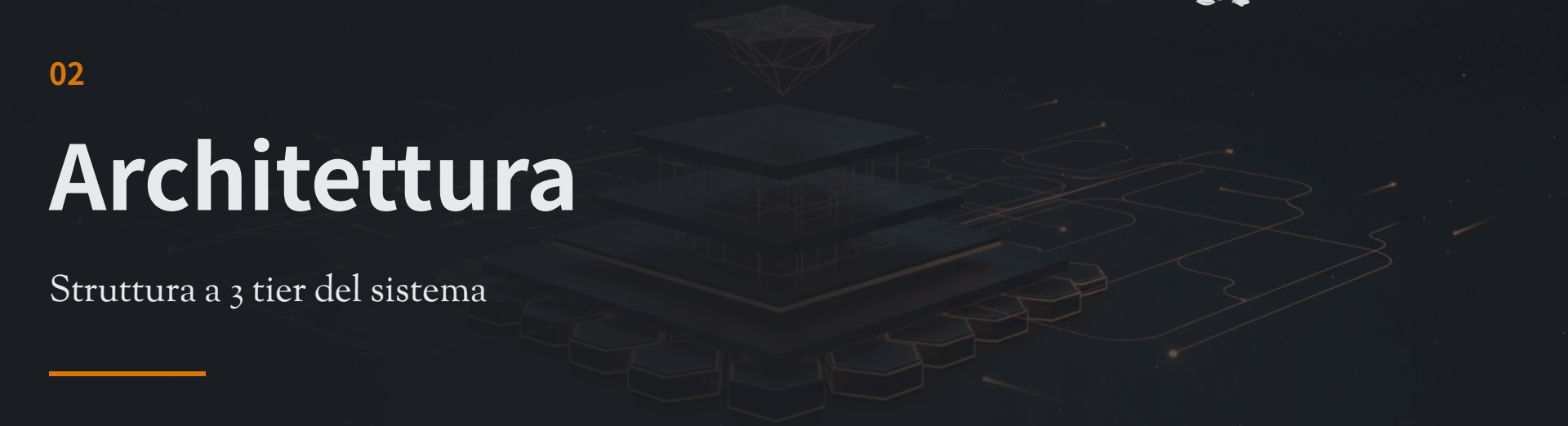
Manutenibilità
Progettazione modulare e documentata

Politecnico di Torino

02

Architettura

Struttura a 3 tier del sistema



System Architecture

Architettura a 3 Livelli

Frontend

Presentation Layer

Interfaccia utente sviluppata con **React** e gestita tramite **Vite** . Comunicazione real-time tramite WebSocket API nativa.

React | Vite | WebSocket | CSS Modules

Backend

Application Layer

Logica applicativa in **Rust** con framework **Axum** . Gestione concorrenza con Tokio, accesso dati con SQLx.

Rust | Axum | Tokio | SQLx | Serde

Database

Data Layer

SQLite in sviluppo per semplicità e portabilità. Struttura compatibile con PostgreSQL per produzione.

SQLite | PostgreSQL | SQLx | Migrazioni

Vantaggi dell'Architettura

- **Separazione responsabilità** tra livelli
- **Facilità di manutenzione** ed estensione
- **Maggiore sicurezza** con isolamento dati
- **Scalabilità orizzontale** dei componenti

Tool di Sviluppo

- Git
- Cargo
- npm

Sicurezza

- Argon2 hashing
- JWT tokens

Politecnico di Torino

03

Frontend

Interfaccia utente con React



Frontend Architecture

Struttura del FrontEnd

Organizzazione del Codice

Il frontend è strutturato seguendo il pattern **component-based** di React, con separazione chiara tra componenti riutilizzabili, pagine e gestione dello stato globale.

components/

LoginForm, InputArea, Navbar...

pages/

LoginPage, ChatPage, InvitesPage

context/

WebSocketContext.jsx

Chat Page

Pagina principale che integra Sidebar, ChatHeader e MessagesContainer per una gestione completa della chat.

- ✓ Auto-scrolling
- ✓ Formattazione nomi
- ✓ Selezione default

Invites Page

Gestione completa del ciclo di vita degli inviti, suddivisi in sezioni "Inviati" e "Ricevuti".

- ✓ Auto-refresh 5s
- ✓ Logica scadenza
- ✓ Layout a colonne

Login Page

Modulo unico e versatile per la gestione di Login e Registrazione, con switch dinamico delle modalità.

- ✓ Modalità duale
- ✓ HTTP API
- ✓ Session storage

Tecnologie

React 18
UI Framework

Vite
Build Tool

React Router
Navigation

WebSocket API
Real-time Comm

Pattern

- Separation of Concerns
- Component Reusability
- Global State Context

State Management

WebSocket Context e Comunicazione

Pattern Context API

Il **WebSocketContext** mantiene un'unica connessione WebSocket attiva e distribuisce lo stato globale a tutte le pagine senza prop drilling.

1. **Riferimento al Socket**
   useRef mantiene la connessione "viva" senza causare re-rendering.

2. **Auto-Login**
   useEffect ripristina sessione da sessionStorage all'avvio.

3. **Centralino handleMessage**
   Switch aggiorna lo stato React in base al type del messaggio.

Stato Globale

- **user**
  Username corrente autenticato
- **groups**
  Array delle chat disponibili
- **messages**
  Oggetto dizionario per group_id
- **invitations**
  Inviti ricevuti e inviati
- **wsError**
  Errori per innescare toast

- **sendMessage**
  Invio messaggi
- **createChat**
  Creazione chat
- **sendInvite**
  Invio inviti
- **respondToInvite**
  Risposta inviti

Politecnico di Torino

04

Backend

Logica applicativa in Rust



Backend Architecture

Pattern Controller-Service-Repository

Separazione delle Responsabilità

Il backend segue il pattern **Controller-Service-Repository** per isolare chiaramente responsabilità e facilitare manutenzione e testing.

Controllers

Gestiscono richieste HTTP/WebSocket, validano dati e orchestrano la logica.

- authentication_controller.rs
- web_socket_controller.rs

Services

Implementano la business logic: validazione, hashing, gestione gruppi.

- user_service.rs
- chat_service.rs
- auth_service.rs

Repositories

Accesso ai dati con query sicure, atomicità e integrità referenziale.

- group_repository.rs
- message_repository.rs
- user_repository.rs

Flusso di una Richiesta

Client
HTTP/WebSocket

→

Controller
Validazione

→

Service
Business Logic

→

Repository
Query SQL

Vantaggi

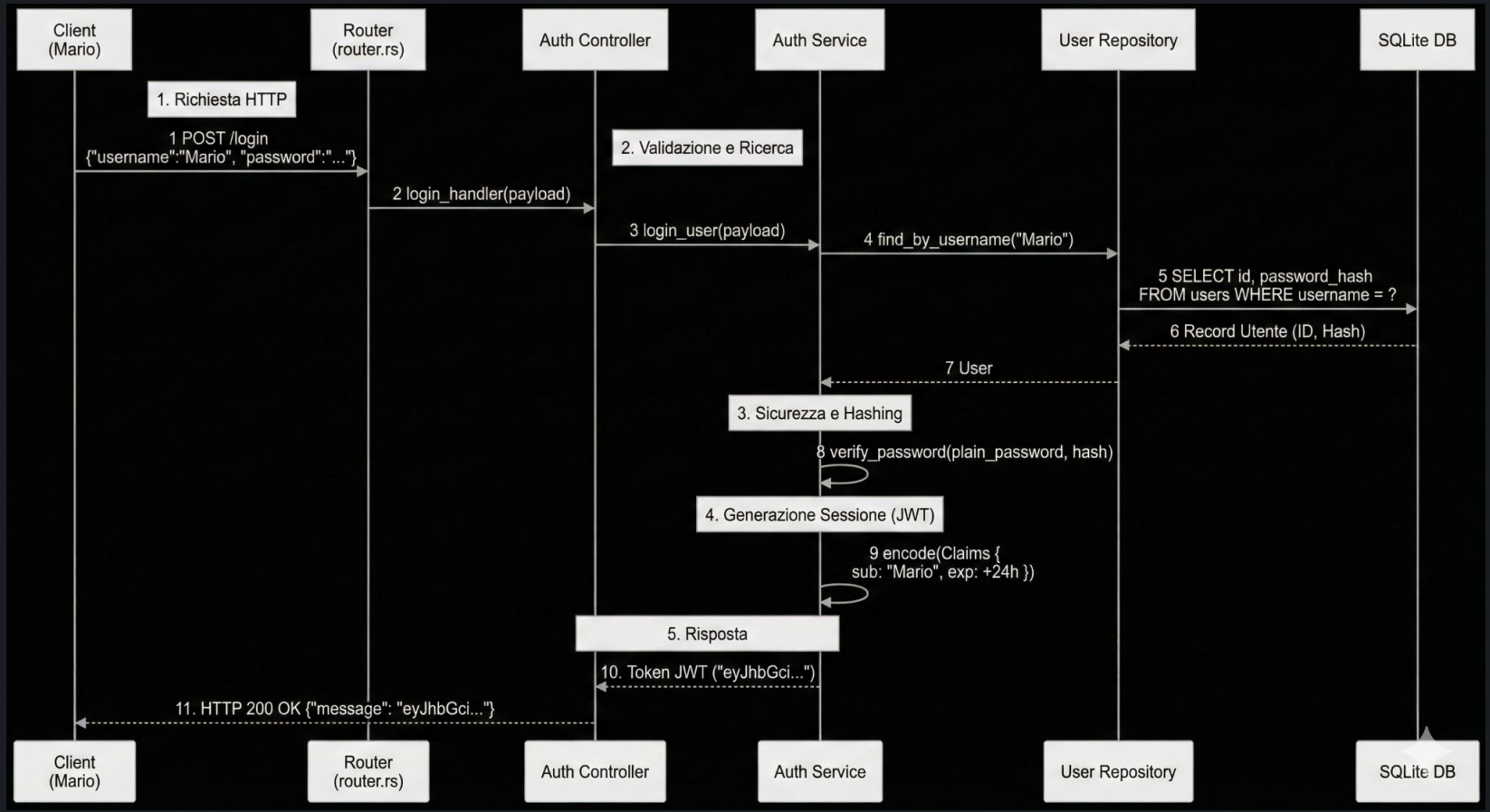
- **Testabilità:** Ogni livello è testabile
- **Manutenibilità:** Modifiche isolate
- **Riusabilità:** Servizi condivisi
- **Scalabilità:** Facile estendere

File Core

- main.rs
- router.rs
- state.rs
- error.rs
- cpu_logger.rs

Authentication Flow

Flusso di Autenticazione



Authentication Flow

Flusso di Autenticazione

1 Richiesta HTTP

Il frontend invia POST a `/login` con JSON contenente username e password.

2 Validazione e Ricerca

Il service valida input e cerca l'utente nel database tramite repository.

3 Verifica Password

Confronto con **Argon2** : la password in chiaro viene processata e confrontata con l'hash salvato.

4 Generazione JWT

Creazione token con **claims** (username, user_id, exp 24h) firmato con chiave segreta.

5 Risposta al Client

HTTP 200 OK con token nel campo message della risposta JSON.

Sicurezza

Argon2 Hashing
Algoritmo memory-hard

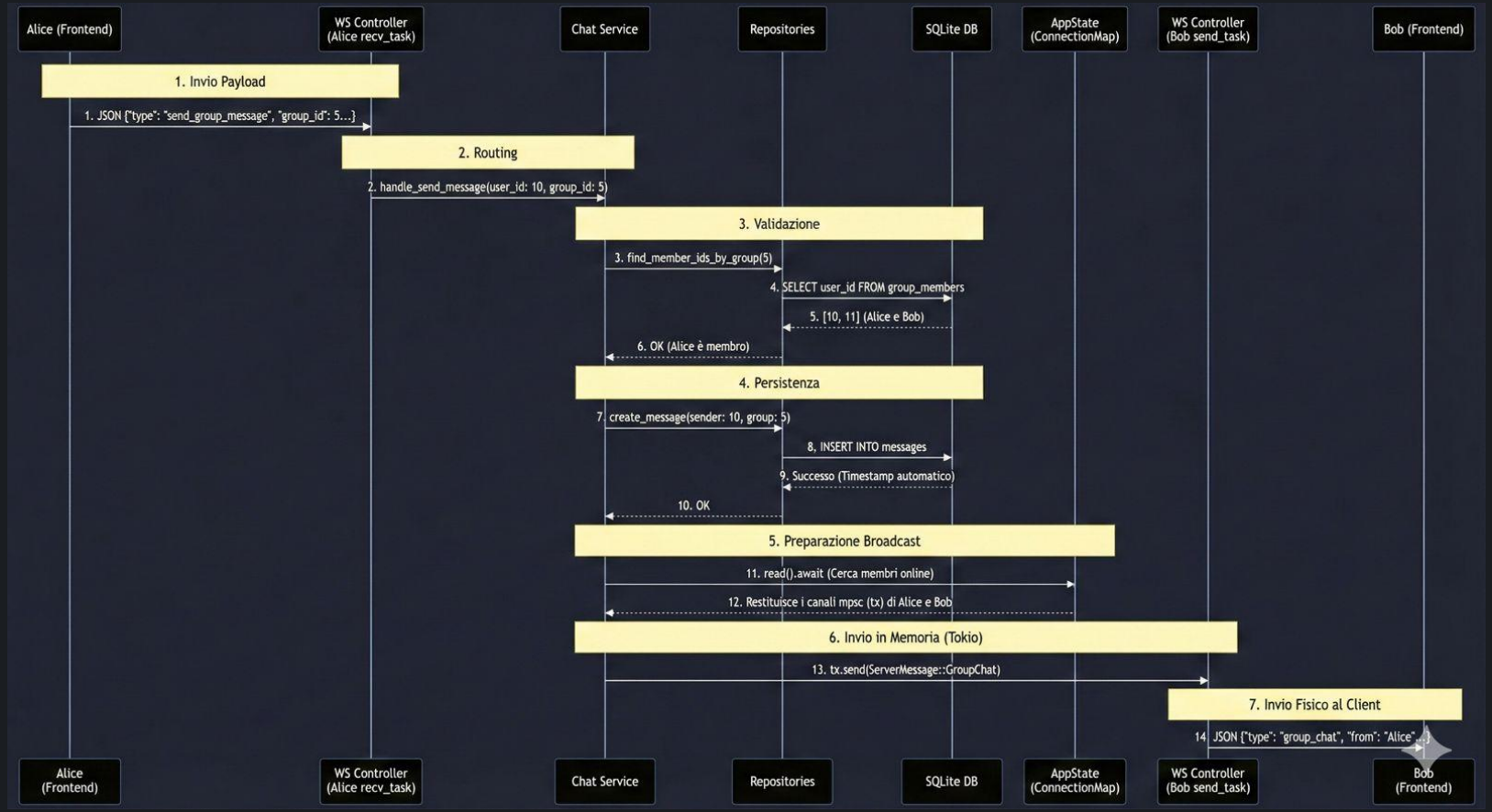
JWT Tokens
Scadenza 24 ore

Messaggi Generici
"Invalid username or password"

JWT Claims

- **sub:** username
- **user_id:** ID utente
- **exp:** scadenza 24h

Flusso dei Messaggi in Tempo Reale



Real-time Messaging

Flusso dei Messaggi in Tempo Reale

1 Frontend Invia Payload

WebSocket invia JSON con type, group_id e content.

```
{ "type": "send_group_message", "group_id": 5, "content": "Ciao!" }
```

2 Controller Riceve e Smista

Deserializzazione con **serde_json** e pattern matching su ClientMessage::SendGroupMessage.

3 Validazione Sicurezza

Il service verifica che il mittente sia membro del gruppo prima di procedere, altrimenti restituisce **ValidationError** .

4 Salvataggio Database

Query SQL parametrizzata INSERT con **timestamp automatico** nel campo created_at.

5 Broadcast ai Membri

Costruzione ServerMessage::GroupChat e broadcast a tutti i membri online tramite **ConnectionMap** .

Performance

Async

Gestione I/O

Vantaggi WebSocket

- Full-duplex
- Low latency
- Stateful connection

Pattern Echo

Il server invia il messaggio anche al mittente per conferma.

Concurrency Management

Gestione Concorrenza e Stato Globale

ConnectionMap - Stato Globale

La **ConnectionMap** è una HashMap thread-safe che traccia tutte le connessioni WebSocket attive, permettendo broadcast efficiente.

```
pub type ConnectionMap = Arc<RwLock<HashMap<i64,HashMap<Uuid, mpsc::Sender<ServerMessage>>>>>;
```

Struttura Dati

i64
User ID dal database

UUID
Per device/sessione

mpsc::Sender
Canale Tokio per invio

Vantaggi

- ✓ Multi-device per utente
- ✓ Broadcast efficiente
- ✓ Thread-safe con RwLock

Tokio Tasks

Sfrutta il runtime **Tokio** per gestire in modo efficiente le operazioni asincrone e la concorrenza.

- ✓ send_task: invio messaggi
- ✓ recv_task: ricezione messaggi
- ✓ CPU logger: ogni 2 min

CPU Logger

Task di background che monitora e registra costantemente l'utilizzo della CPU del server.

- Intervallo: 120 secondi
- Cross-platform
- Non blocca chat

```
[2026-02-16 17:03:33] Server CPU Usage: 0.16387% | CPU Time Usage: 0.010s
[2026-02-16 17:05:33] Server CPU Usage: 0.71490% | CPU Time Usage: 0.863s
[2026-02-16 17:07:33] Server CPU Usage: 0.36340% | CPU Time Usage: 0.434s
[2026-02-16 17:09:33] Server CPU Usage: 0.03788% | CPU Time Usage: 0.046s
[2026-02-16 17:11:33] Server CPU Usage: 0.03928% | CPU Time Usage: 0.037s
[2026-02-16 17:12:22] Server CPU Usage: 0.08615% | CPU Time Usage: 0.006s
[2026-02-16 17:14:22] Server CPU Usage: 0.03654% | CPU Time Usage: 0.056s
```



Politecnico
di Torino

05

Database

Persistenza e gestione dati

Database Schema

Schema Database e Tabelle

users

Memorizza identità e credenziali degli utenti.

- **id:** PK autoincrement
- **username:** univoco
- **password_hash:** Argon2

groups

Definisce gruppi e chat private (DM).

- **id:** PK autoincrement
- **name:** opzionale
- **is_dm:** boolean

group_members

Relazione Many-to-Many tra utenti e gruppi.

- **group_id:** FK → groups
- **user_id:** FK → users
- **PK:** (group_id, user_id)

messages

Archivia tutti i messaggi scambiati.

- **id:** PK autoincrement
- **group_id:** FK → groups
- **sender_user_id:** FK → users
- **content:** testo
- **created_at:** timestamp

invitations

Gestisce richieste di accesso ai gruppi o DM.

- **id:** PK autoincrement
- **from_user_id:** FK → users
- **to_user_id:** FK → users
- **group_id:** FK → groups
- **status:** pending/accepted/rejected
- **created_at:** timestamp
- **updated_at:** timestamp

Tecnologie

SQLite
Sviluppo: semplice e portatile

SQLx
Query asincrone e type-safe

Migrazioni
Versionate e tracciabili

Data Security

Sicurezza e Integrità dei Dati

Meccanismi di Protezione

Il database implementa multiple strategie di sicurezza per garantire integrità, coerenza e protezione dai principali attacchi.

Chiavi Esterne

Vincoli FK per coerenza referenziale tra tabelle.

ON DELETE CASCADE

Cancellazione automatica dei record correlati.

Query Parametrizzate

Prevenzione SQL injection con prepared statements.

Migrazioni Versionate

Tracciamento modifiche schema database.

Principi ACID

- **A** **Atomicity** — Tutto o niente
- **C** **Consistency** — Stato valido
- **I** **Isolation** — Transazioni indipendenti
- **D** **Durability** — Persistenza garantita

Esempio: Query Parametrizzata

```
// SQLx prepared statement sqlx::query( "INSERT INTO messages (group_id, sender_user_id, content) VALUES (?, ?, ?)") .bind(group_id) .bind(sender_id) .bind(content) .execute(&mut *tx) .await?;
```

I parametri vengono sanitizzati automaticamente.

Vantaggi SQLx

- ✓ Compile-time checked
- ✓ Async/await native
- ✓ Type-safe queries
- ✓ Pooling integrato

Compatibilità

Schema SQL compatibile SQLite ↔ PostgreSQL.

Politecnico di Torino

06

Esperienza Utente

Funzionalità dell'applicazione

Core Features

Funzionalità Principali

Global Chat

Stanza pubblica dove tutti i nuovi utenti vengono inseriti automaticamente al momento della registrazione.

- Accesso immediato

Direct Messages

Canale di comunicazione esclusivo e bidirezionale (uno-a-uno) tra due utenti.

- ✓ Richiede invito
- ✓ Nome interlocutore

Gruppi

Spazi di discussione multicanale con nome personalizzato e gestione membri.

- ✓ Nome personalizzato
- ✓ Invito membri

Sistema Inviti

Gestione completa del ciclo di vita degli inviti con stati pending, accepted, rejected.

- ✓ Protezione anti-spam
- ✓ Cooldown 2 min

Autenticazione

Registrazione
Username e password

Login
JWT token 24h

Sicurezza
Argon2 hashing

Privacy First

- **Nessun accesso automatico:** tutte le conversazioni richiedono invito esplicito.
- **DM privati:** solo due partecipanti, nessuna espansione.
- **Visibilità controllata:** chat appaiono solo dopo accettazione.

Anti-Spam

- Cooldown 2 min post-rifiuto
- Blocco re-invio immediato
- Pulizia automatica inviti

Compatibilità

Multipiattaforma

User Interface

Interfaccia e Tempo Reale

Componenti UI

Navbar Superiore
Navigazione tra "Le Mie Chat" e "Inviti" con badge notifiche

Sidebar Laterale
Elenco conversazioni con anteprima ultimo messaggio

Area Messaggi
Visualizzazione differenziata sent/received con auto-scroll

Input Area
Campo testo con validazione e invio tramite Enter

Sistema di Notifiche

Toast Success
"Login effettuato!", "Messaggio inviato", "Gruppo creato"

Toast Error
"Credenziali non valide", "Utente non trovato"

Toast Info
"Invito inviato", "Nuovo messaggio"

Badge Notifiche
Indicatore numerico rosso su "Inviti" per richieste pendenti

Tempo Reale
Nessun refresh manuale

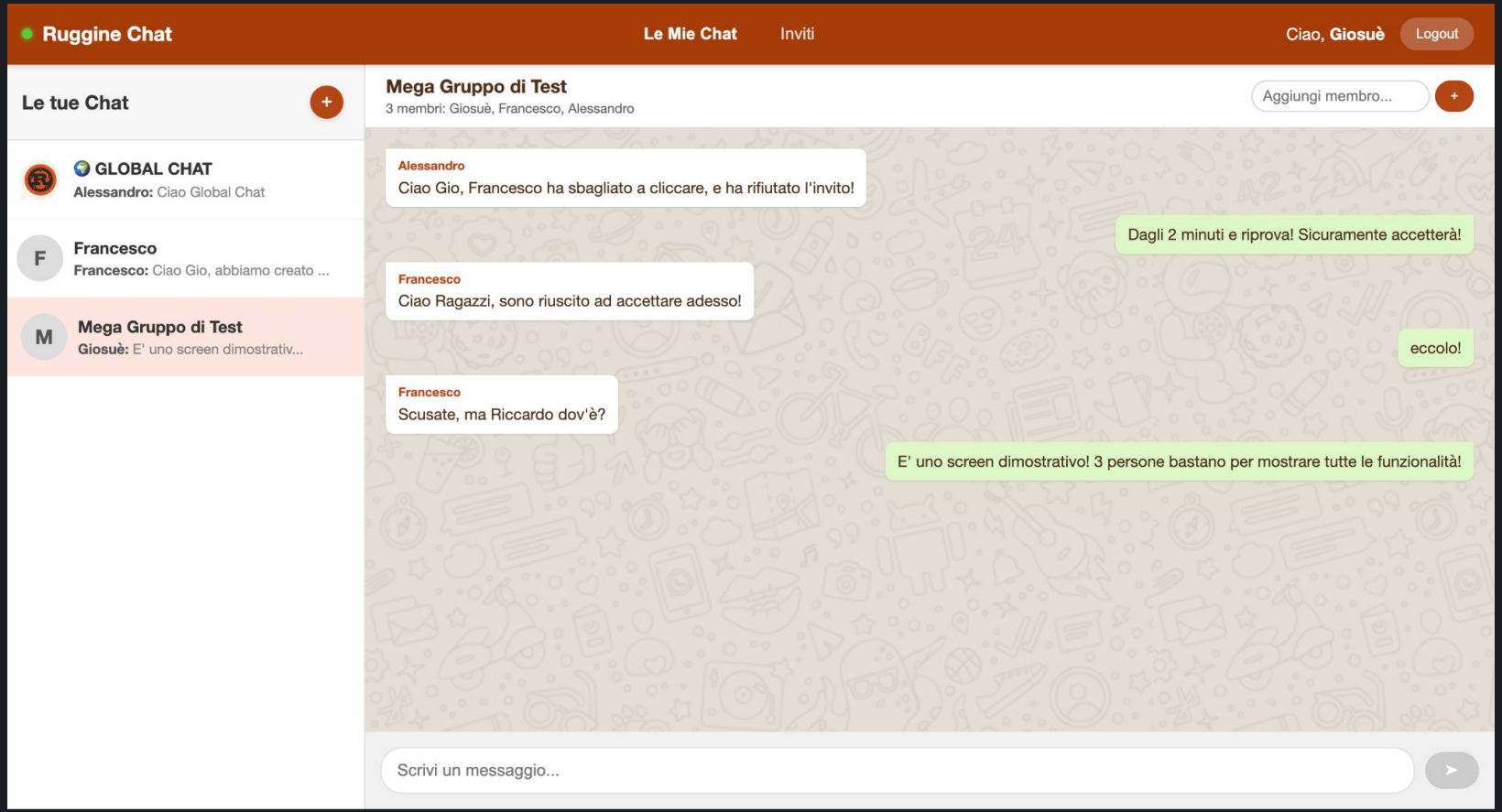
Auto-Scrolling
Focus su ultimi messaggi

Anteprime Live
Ultimo messaggio sidebar

Indicatore Connessione
Icona verde accanto nome

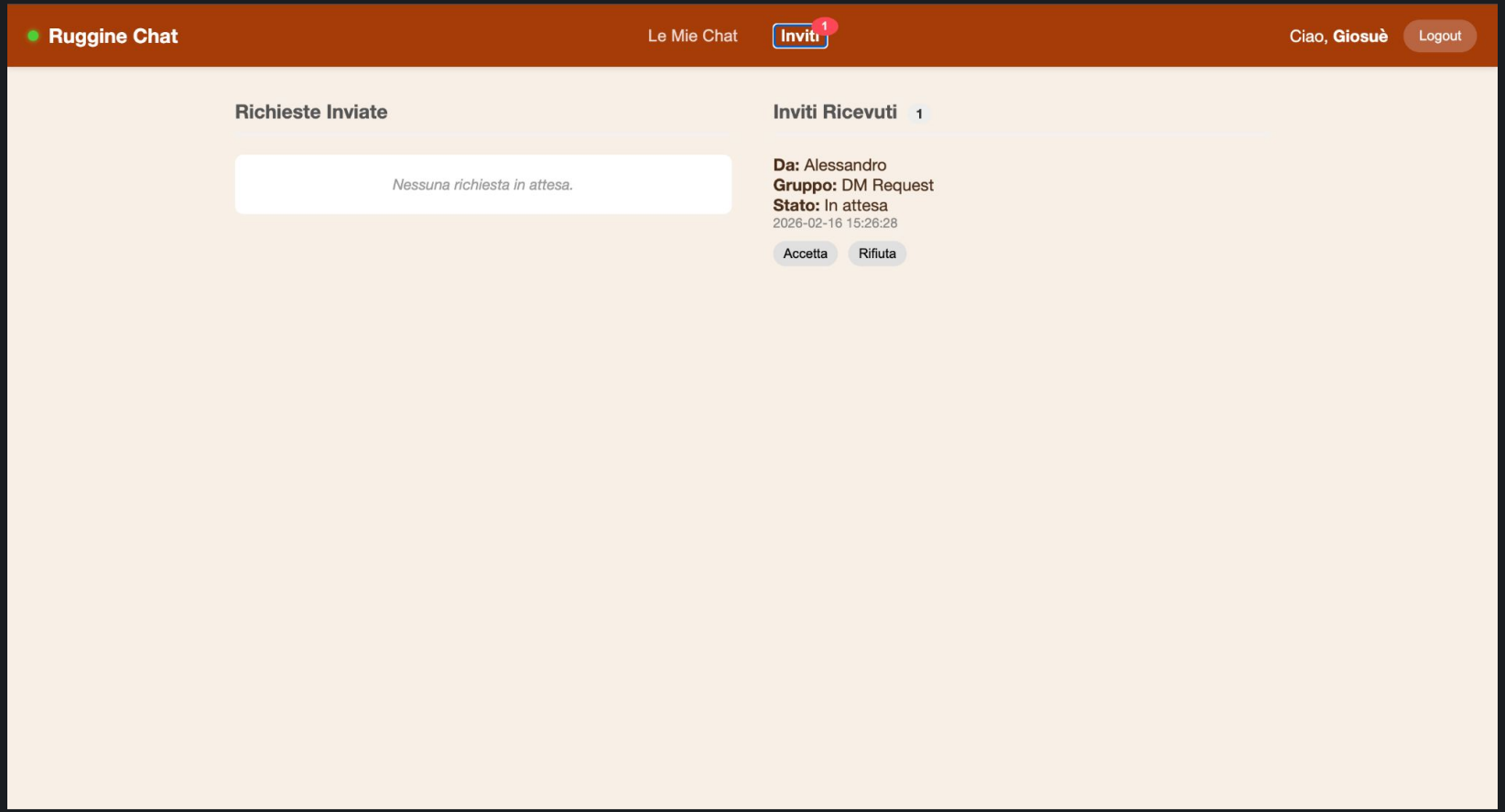
Core Features

Chat Page



Core Features

Invitation Page





Politecnico di Torino

07

Sviluppi Futuri

Miglioramenti e nuove feature

Future Roadmap

Proposte di Evoluzione

Migliorie Architetturali

Microservizi

Separazione autenticazione, chat e notifiche in servizi distinti per scalabilità indipendente.

Containerizzazione

Docker Compose per avvio completo con un comando, isolamento dipendenze.

CI/CD Pipeline

Automazione test, build e deploy con GitHub Actions o GitLab CI.

Nuove Feature

Notifiche Push

Firebase Cloud Messaging per notifiche su smartphone quando l'app è in background.

Supporto Mobile

Progressive Web App (PWA) per uso offline e installazione su smartphone senza store.

Ruoli e Permessi

Admin, moderatore, membro con permessi granulari e strumenti di moderazione.

Integrazioni Esterne

OAuth (Google Login), cloud storage (Google Drive), analytics (Google Analytics).

Security Considerations

Riflessioni sulla Sicurezza

Sistema JWT - Punti di Forza

I **JSON Web Tokens** gestiscono sessioni in modo stateless. La firma crittografica garantisce che il token non sia stato modificato.

Struttura JWT

Header: algoritmo e tipo
Payload: claims (sub, user_id, exp)
Signature: HMAC-SHA256

Token Theft - Rischi

Il payload JWT è **leggibile** (Base64). Se intercettato, un attacker può impersonare l'utente fino alla scadenza.

Rischio
XSS o traffico non criptato → furto token

Protezione
Chiave segreta server previene forging

Best Practices

- **HTTPS/WSS:** connessioni sempre criptate
- **Short expiration:** token con scadenza breve
- **Refresh tokens:** rotazione periodica
- **HttpOnly cookies:** protezione XSS

Certificati TLS

Per HTTPS serve un certificato ufficiale rilasciato da una Certification Authority (CA).

Let's Encrypt Cloudflare

Implementazione Attuale

- ✓ Token in chiaro (sviluppo)
- ✓ Scadenza 24 ore
- ✗ HTTPS non implementato



Grazie per l'attenzione

Domande?

Progetto Ruggine – Gruppo 50

Alessandro Annecchini	Riccardo Marconi	Francesco Palmisani	Giosuè Pinto
s343760	s342227	s343429	s342711

Corso di Programmazione di Sistema 2025/2026



Politecnico di Torino